@vjcloudops

ANSIBLE

INTERVIEW QUESTIONS & ANSWERS

Crack Interviews with Real-World Ansible Scenarios and Explanations

Q1: What is Ansible, and why is it widely used for automating IT infrastructure management?

Answer:

Ansible is an open-source automation tool used to configure systems, deploy applications, and manage infrastructure as code (IaC). It's agentless and works over SSH (for Linux) or WinRM (for Windows), making it easy to start with and ideal for scaling automation across hybrid environments.

Why it's popular:

- **Simple YAML syntax** – No programming skills required.
- **Agentless** – No need to install software on target nodes.
- **Idempotent** – Running a playbook multiple times results in the same system state.
- **Modular** – Rich ecosystem of pre-built roles and modules.

Real-life analogy:
Think of Ansible as your personal house manager. You just write a clear instruction sheet (playbook) like:

> "Install AC, check the fridge, paint the walls blue,"
> and every room (server) gets exactly that setup—no more, no less.

Example use cases:

- Provisioning web servers (Apache, Nginx)
- Installing packages and patching OS
- Managing user accounts and SSH keys
- Deploying code in test/production

Q2: What's the difference between using Ansible for "repair" and Terraform for "replace"? How do they complement each other?

Answer:

Both Ansible and Terraform are IaC tools, but they serve different purposes and follow different philosophies in infrastructure automation:

Ansible = Repair
Ansible is great at **configuring and maintaining** systems. If something drifts (like a package version), it **repairs** it by bringing the system back to the desired state without rebuilding it.

Terraform = Replace
Terraform is a declarative tool used to **provision infrastructure**. It maintains a state file and will **replace** resources if they drift away from the desired config.

Analogy:

- Terraform is like constructing a new building from a blueprint. If something major goes wrong, you might demolish and rebuild.
- Ansible is like a repair crew - they come in to fix the plumbing, install AC, or repaint walls without tearing down the structure.

How they work together:

- Use **Terraform** to provision your EC2 instances, load balancers, and networks.
- Use **Ansible** to install packages, configure services, deploy apps on those instances.

Example:

```
# Terraform
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"
}

# Ansible
- name: Install and start nginx
  hosts: web_servers
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
```

Q3: What command would you use to check for indentation errors in Ansible playbooks? How can incorrect indentation affect playbook execution?

Answer:

YAML (used in Ansible playbooks) is **strictly indentation-sensitive**. Even a single misplaced space can break your automation or make it behave unpredictably.

Command to check indentation and syntax:

```
ansible-playbook your-playbook.yml --syntax-check
```

This command **does not run** the playbook - it simply validates the YAML structure and Ansible logic.

Why it matters:
Indentation in YAML defines hierarchy. If it's off, your task might:

- Fail silently
- Be skipped
- Apply to the wrong block

Analogy:
Think of YAML like a family tree -> indentation shows relationships. If you mess up the levels, your uncle might become your child, and the whole structure collapses!

Example of incorrect vs correct indentation:

❌ Bad:

```
- name: Install packages
  apt:
  name: nginx
  state: present
```

✅ Good:

```
- name: Install packages
  apt:
    name: nginx
    state: present
```

Q4: Why is it important to check the syntax of a playbook before executing it in Ansible? What command helps you perform this check?

Answer:

Checking syntax before execution is crucial because Ansible doesn't compile like traditional code - it directly connects to your infrastructure. A small mistake can have real-world consequences like service downtime or misconfigured systems.

Command to check syntax:

```
ansible-playbook playbook.yml --syntax-check
```

This is like a **dry-run for errors** - it validates:

- YAML structure
- Task-level syntax
- Role paths, variable usage

Benefits:

- Prevents accidental changes on production
- Catches typos early
- Saves debugging time

Analogy:
Imagine launching a rocket without running a system check first. One missed variable, and boom - there goes your server.

Best practice:

Always run `--syntax-check` before deploying to **production or staging**.

Q5: How does running a dry run in Ansible help test playbooks before they affect real systems? What benefits does it provide to users?

Answer:

A dry run in Ansible is like a rehearsal before the real performance. It lets you preview what changes would be made without actually making them.

Command:

```
ansible-playbook playbook.yml --check
```

This tells Ansible to simulate the execution and show which tasks would run or change something - but nothing is modified on the target machine.

Benefits:

- Prevents accidental configuration drift
- Lets you validate logic and scope of changes
- Boosts confidence before deploying in production
- Saves time in debugging by catching potential issues earlier

Analogy:
Imagine you're assembling IKEA furniture. A dry run is like laying out all the parts and reading the manual without using screws yet it helps you avoid putting the wrong leg on the wrong side!

Tip:

Use `--check` along with `--diff` to see exactly what changes would occur in file content.

Q6: How do you debug a failed Ansible playbook? What tools or strategies would you use to identify and resolve issues?

Answer:

Debugging a failed Ansible playbook requires a combination of logs, verbosity, and smart module usage. Here's a structured way to approach it:

Steps to debug:

1. **Increase verbosity:**

```
ansible-playbook playbook.yml -vvv
```

   The more `v`'s you add, the more detailed output you get.

2. **Use the `debug` module** in playbooks:

```
- debug:
            var: my_variable
```

3. **Add `when` clauses** to avoid conditional errors.

4. **Check return codes, error messages, or register results** for logic errors.

Best practices:

- Use `--syntax-check` before running.
- Test in staging first.
- Break large playbooks into small, testable roles.

Analogy:

Debugging a playbook is like fixing a recipe. If the cake fails, you increase the oven light (`-vvv`), test the batter (`debug`), and rerun only the parts that went wrong.

Q7: What role do inventory files play in Ansible, and how do they help automate tasks across multiple servers and environments?

Answer:

Inventory files are the heart of Ansible's infrastructure awareness. They define which machines Ansible will manage, organized into groups, like production, dev, or web servers.

Basic example (INI format):

ini

```
[web]
web01 ansible_host=192.168.1.10
web02 ansible_host=192.168.1.11

[db]
db01 ansible_host=192.168.1.20
```

Why it matters:

- Allows targeting **specific groups** or **all hosts**
- Supports **dynamic inventories** (AWS, Azure, GCP)
- Enables **role-based tasks** (e.g., only apply DB updates to `db` group)

Analogy:
Think of an inventory file as your **contact list**. Instead of calling every person manually, you just group contacts ("family", "friends", "colleagues") and send a group message. Ansible does the same with your servers.

Q8: What are Ansible modules and how do they simplify the automation of tasks like package installation and service management? Can you provide an example?

Answer:

Modules are reusable, standalone scripts that Ansible calls to perform specific tasks like installing packages, managing users, or restarting services.

Why modules are powerful:

- They are **idempotent** ; no duplication or conflicts
- Abstract system differences (e.g., `apt` vs `yum`)
- Simplify complex tasks with just a few lines

Common module example:

```
- name: Install nginx
  apt:
    name: nginx
    state: present
```

This tells Ansible to:
Check if nginx is installed
Install it only if it's not present

Analogy:
Think of a module like a Lego brick -> you don't need to shape it yourself. Just plug it into your playbook and build up your automation.

Popular modules:

- `apt`, `yum`, `dnf` – for package management
- `service` – to start/stop services
- `user` – to manage Linux users
- `copy`, `template` – to manage files

Q9: What is the purpose of using ad-hoc commands in Ansible, and how would you use them to check disk space across multiple hosts?

Answer:

Ad-hoc commands are quick, one-line Ansible commands you run from the terminal to perform simple tasks without writing a full playbook.

Use cases:

- Check service status
- Reboot machines
- Run commands like checking disk usage, uptime, etc.
- Install a package quickly

Example: Check disk space on all hosts:

```
ansible all -m shell -a "df -h"
```

- `all`: target group from the inventory
- `-m shell`: use the shell module
- `-a`: command to execute

Analogy:
Think of ad-hoc commands as the WhatsApp voice note of Ansible: quick, effective, and doesn't require the effort of writing a full email (playbook)!

Q10: How do you copy files from the Ansible control machine to client machines using Ansible? Provide an example with a file named file.txt.

Answer:

Ansible's **copy** module helps you transfer files from the control node (your machine) to any managed host.

Example:

```
- name: Copy file.txt to remote host
  hosts: webservers
  tasks:
    - name: Copy file
      copy:
        src: /home/ansible/file.txt
        dest: /tmp/file.txt
```

- `src`: path on the control machine
- `dest`: where to place it on the remote host

Tip: This works well for configuration files, certificates, scripts, etc.

Example:

> Want to send config files from your laptop to 10 servers?
> Just use the **`copy`** module in Ansible and skip SCP and loops.

Q11: Explain how the `remote_src` option works when copying files between two remote machines. Provide a use case.

Answer:

When using the **`copy`** module, by default Ansible looks for **`src`** on the control machine. But if you set **`remote_src:yes`**, Ansible treats **`src`** as a path on the remote host instead.

Use case:
You want to copy a file already present on one server to another location on the **same server** (or across remotes via delegate):

```
- name: Copy a file on the same host
  copy:
    src: /etc/config/app.conf
    dest: /backup/app.conf
    remote_src: yes
```

Why it's useful:

- Avoids fetching from control node
- Useful in **backup**, **migrations**, or when using **delegation**

Analogy:
You're inside a warehouse (remote server), and instead of bringing an item from HQ (control node), you just shift it from shelf A to shelf B within the same warehouse. That's **remote_src**.

Example:
Use **remote_src: yes** to move files already present on the server itself - no need to bring them to the control machine.

Q12: What is the command used to register the output of a task in Ansible, and how do you access the output in a subsequent task?

Answer:

You use the **`register`** keyword to capture the output of a task and store it in a variable for later use.

Example:

```
- name: Get hostname
  shell: hostname
  register: hostname_output

- name: Show output
  debug:
    var: hostname_output.stdout
```

`hostname_output` stores:

- `stdout`: actual result
- `stderr`: error (if any)
- `rc`: return code
- `stdout_lines`: output in list form

Analogy:
Think of **`register`** as taking a screenshot of a command's output. You can then show or act on it in the next scene (task).

Example:

Want to reuse a command's output in the next task?
Use **`register`** like this:

```
- shell: uptime
  register: result
```

Then access: `result.stdout`

Q13: How can you use the `register` keyword to store the output of a command and use it in a conditional check in Ansible?

Answer:

You can combine **`register`** with **`when`** to make decisions based on a task's result.

Example: Restart service only if file is found:

```
- name: Check if config file exists
  shell: test -f /etc/myapp/config.yaml
  register: config_check
  ignore_errors: true

- name: Restart app
  service:
    name: myapp
    state: restarted
  when: config_check.rc == 0
```

- `rc == 0` means command succeeded (file exists)

Analogy:
You flip a switch (register) and read the light's color (rc). Based on whether it's green, you proceed to the next action (when).

Example:

Conditional automation?
Capture output with **register** and decide actions with **when**.

Q14: How would you create a custom condition using Ansible's registered variables to trigger a task only when specific conditions are met?

Answer:

You can use **register** and write custom conditions using **when**, combining variables and logic.

Example: Run a task only if load average is high:

```
- name: Get system load
  shell: uptime | awk '{print $(NF-2)}'
  register: load_avg

- name: Alert admin if load is high
  debug:
    msg: "High load detected!"
  when: load_avg.stdout | float > 5.0
```

This uses Jinja2 filters (`| float`) to convert output before comparison.

Tip: You can combine multiple conditions using `and`, `or`, etc.

> Playbooks that react to conditions? That's real automation!
> Store output with `register` and use logic like:

```
when: my_var.stdout | float > 5
```

Q15: Why would you use ad-hoc commands in Ansible instead of playbooks, and what are the advantages?

Answer:

Ad-hoc commands are great for quick, one-time tasks that don't require reuse or complex logic. They're perfect for:

- Checking system status
- Restarting services
- Applying updates
- Running shell commands across a fleet

Playbooks vs Ad-hoc:

- Use playbooks for repeatable, well-defined automation.
- Use ad-hoc for quick fixes and fire-and-forget actions.

Analogy:
Playbooks are like scheduled meetings with agendas. Ad-hoc commands? Just a quick text: "Restart Apache now!"

Example:

```
ansible web -m service -a "name=apache2 state=restarted"
```

Sometimes you don't need a playbook.
Just one-liners like:

```
ansible all -m shell -a "uptime"
```

Q16. How can you use Ansible's register feature to capture the status of a service and conditionally restart it?

Using Ansible's **register** feature, you can capture the status of a service, and then use a conditional (**when**) to restart the service only if it's not running. This avoids unnecessary restarts and makes your playbook idempotent.

Scenario:

You want to check if nginx is running, and restart it only if it's not active.

playbook:

```
- name: Check and restart nginx if not running
  hosts: web
  become: yes
  tasks:

    - name: Check nginx service status
      shell: systemctl is-active nginx
      register: nginx_status
      ignore_errors: yes

    - name: Restart nginx if it is not running
      service:
        name: nginx
        state: restarted
      when: nginx_status.stdout != "active"
```

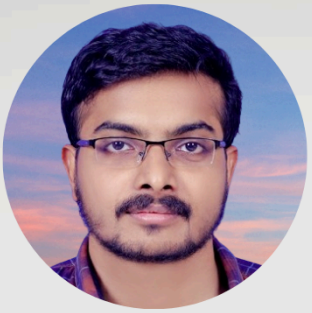
Explanation:

- `shell: systemctl is-active nginx` → Returns `active`, `inactive`, `failed`, etc.
- `register: nginx_status` → Captures the result.
- `when: nginx_status.stdout != "active"` → Only restarts nginx if it's not running.

- `ignore_errors: yes` → Prevents playbook failure if the service is not found or already inactive.

Real-World Analogy:

It's like checking if your car engine is on before turning the key. If it's already running, you don't try to start it again.



Vijesh

Cloud & DevOps Engineer

Share your Thoughts in the comment section

Save the post for reference

careerbytecode.substack.com

@vjcloudops